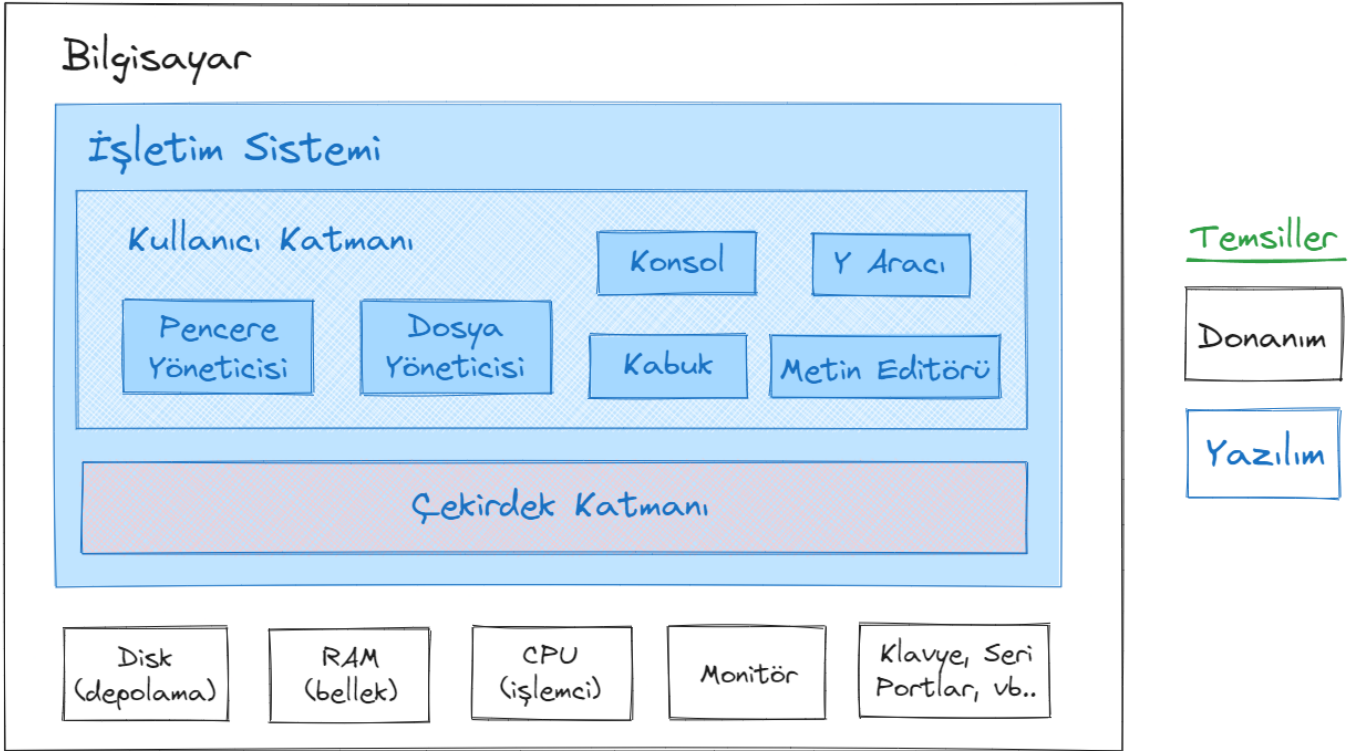


Linux Sistem Mimarisine Giriş

Gerekli çalışma ortamını kurduğumuza göre artık sistemi daha yakından tanıyıp uygulama yapmaya başlayabiliriz. Tüm detayları ile olmasa da kullanacağımız sistemin arkaplanda nasıl çalıştığı ve nasıl bir yapıya sahip olduğu konusunda fikir sahibi olmamız, ileride yapacağımız tüm işlemleri çok daha bilinçli şekilde gerçekleştirebilmemizi sağlayacak. Yani sistemi yönetirken aşağı yukarı arkaplanda neler olduğu konusunda da fikir sahibi olacağız. Bu sebeple bu bölümdeki anlatımlarda Linux sisteminin genel yapısını temel düzeyde kavramayı amaçlıyoruz.

Linux'un ne olduğundan bahsettiğimiz bölümde bilgisayarı ve içindeki işletim sistemini bu şekilde soyutlamıştık hatırlarsanız.

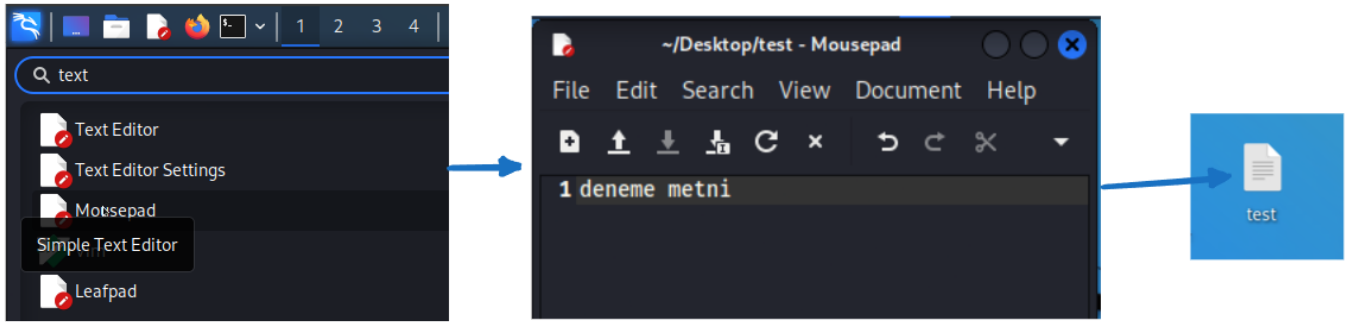


Şimdi tekrar bu şemaya dönecek olursak bakın temelde bir işletim sisteminde “kullanıcı” ve “çekirdek” olmak üzere iki temel alan bulunuyor. Biz kullanıcı alanındaki araçları kullandığımızda, buradaki araçlar çekirdek ile iletişime geçip ilgili görevin çekirdek tarafından donanıma doğru şekilde ifade edilip yaptırılmasını sağlıyorlar. Bu durumdan zaten daha önce de bahsettik. Şimdi çekirdek bölümünü bir kenara bırakıp kullanıcı alanına odaklanmak istiyorum.

Kullanıcı Alanı(User Space-User Mode)

Standart bir kullanıcı olarak sistem üzerinde herhangi bir işi yaparken çok fazla çaba sarf etmek istemeyiz. Örneğin ben bir metin belgesi oluşturmak istiyorum diyelim. Bunun için tek yapmam gereken metin editörünü bulup çalıştırmak ve açılan programa istediğim yazıları yazıp metin belgesini kaydetmek. Yani bu işin bilgisayar donanımına nasıl ifade edileceğini düşünmem gerekmez, ben yalnızca ilgili aracı bulup kullanırım. İşte bu ilgili aracı bulma ve kullanma aşamalarını gerçekleştirebilmek için de, biz kullanıcılara bir çalışma ortamı gerekiyor.

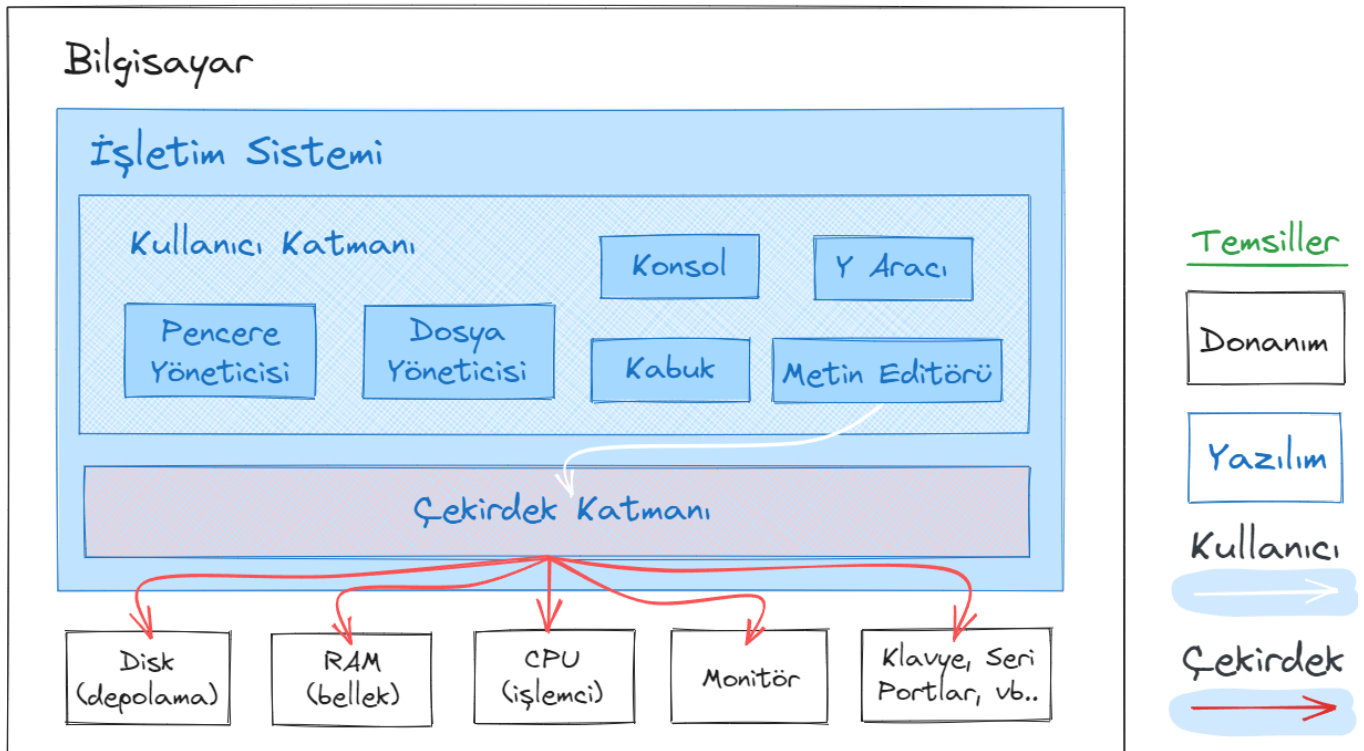
Bizzat metin belgesi oluşturma örneğini ele alacak olursak; öncelikle grafiksel arayüzden metin editörü aracını bulup çalıştırmam daha sonra istediğim metni yazıp dosyayı kaydetmem yeterli. Bu sayede grafiksel arayüz üzerinden metin belgesi oluşturma amacımıza ulaşmış oluyoruz.



Peki bu işimizi nasıl yerine getirdik ? Tabii ki grafiksel arayüzü kullanarak aracımızı bulduk ve çalıştırdık.

Yani aslında bu örnekte bizzat gördüğümüz gibi, ben kullanıcı olarak bir metin belgesine yazı yazmak istediğimde donanıma nasıl iş yaptıracağımı falan düşünmem gerekmez, gerekmemelidir zaten. Ben sadece programı bulup çalıştırdıktan sonra klavyemi kullanarak yazımı yazarım gerisini işletim sistemine bırakırım. İşletim sistemi de bir önceki başlık altında açıkladığımız şekilde tüm gerekli işlemleri katmanlı yapısı sayesinde organize eder ve donanıma bu işi yaptırır.

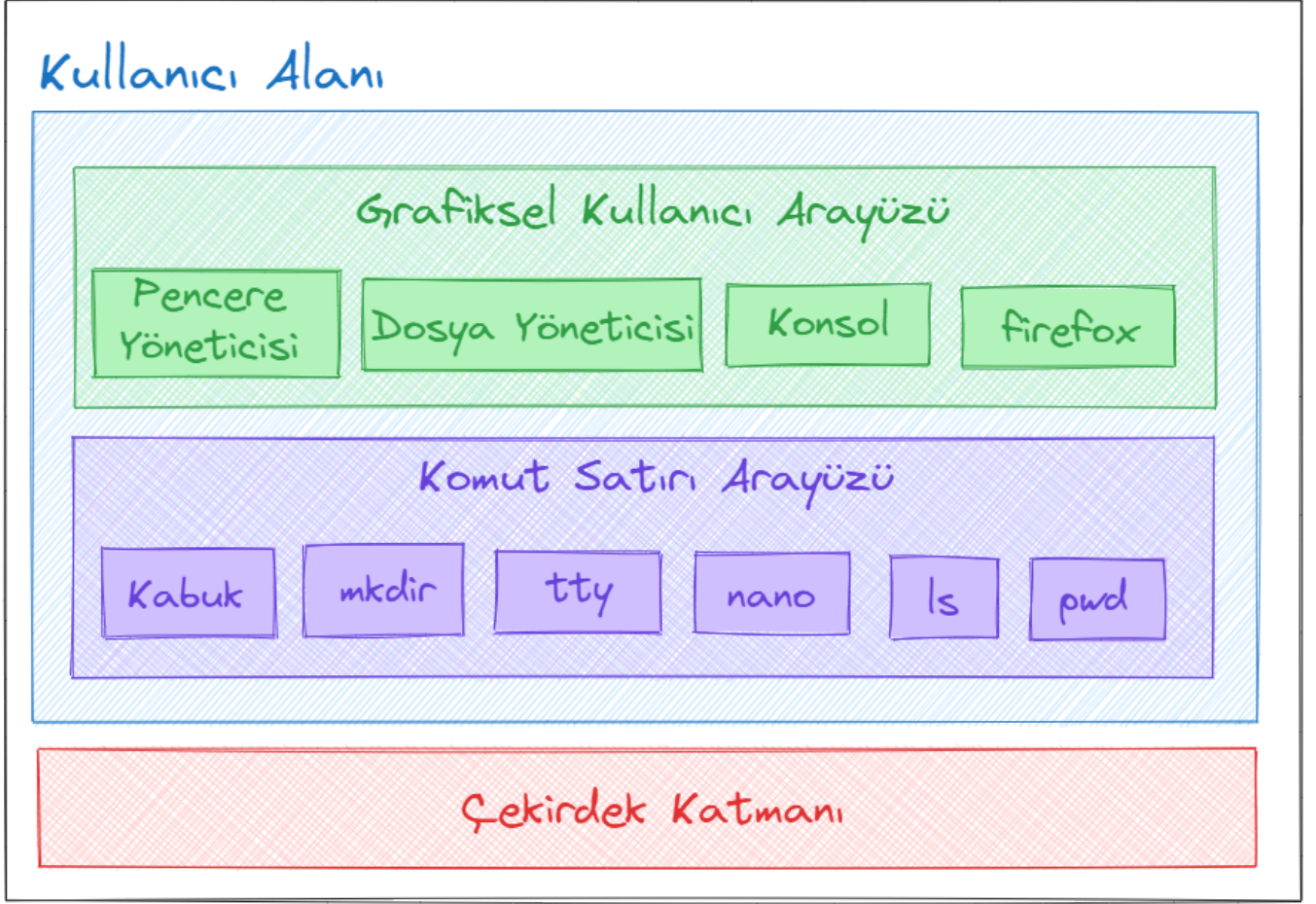
İşte bizim grafiksel arayüzlü menüleri ve metin editörünü kullanırken yaptığımız şey aslında kullanıcı alanındaki araçları kullanarak sisteme emirler vermektir. Daha önce bahsetmiş olduğum kullanıcı alanı işte tam olarak burası.



Yani kullanıcı alanına aslında komuta kontrol alanı da diyebiliriz. Nitekim ben metin editörünü bulup çalıştırırken ve yazı yazarken aslında grafiksel arayüze sahip kullanıcı alanında çalıştım. Ayrıca henüz bahsetmedik ancak tıpkı grafiksel arayüz ortamı gibi kullanıcıların sistemi yönetmek için kullandıkları komut satırı arayüzü de bulunuyor. Yani aslında kullanıcı alanı olarak tabir ettiğimiz soyut katmanın kendisi de

"grafiksel kullanıcı arayüzü(GUI)" ve "komutu satırı arayüzü(CLI)" olmak üzere iki tür arayüz ortamını barındırıyor.

İşletim Sistemi



Şimdi burada bahsedilenleri daha iyi anlamak adına anlatımlarımıza sırasıyla bu ortam arayüzlerinden bahsederek devam edebiliriz.

Grafiksel Kullanıcı Arayüzü

Grafiksel arayüzün ne olduğunu tanımlayarak başlayacak olursak. Grafiksel arayüzden kastımız genellikle fare kullanarak, bir takım tıklama ve sürükleyip bırak işlemleri ile sistemi yönetebildiğimiz görselliğin ön planda olduğu çalışma ortamıdır. Bu ortam grafiksel arayüze sahip olan araçların çalıştırılabilmesi için mevcut. Dolayısıyla grafiksel arayüze sahip araçları kullanabilmek için bu ortama ihtiyacımız var. Örneğin ofis yazılımlarını kullanabilmek için grafiksel arayüz ortamına ihtiyacımız var. Çünkü ofis yazılımı tasarımı gereği grafiksel arayüzden yönetilebilecek yapıda tasarlanmış bir araçtır. Bizler ofis yazılımını kullanırken, butonlar ve çeşitli menülere tıklayarak aracı yönetiriz. Bizim bu butonları görüp üzerlerine tıklayabilmemizi sağlayan ortam da işte grafiksel kullanıcı arayüzüdür.

Sanırım grafik arayüzü yeterince açık bir şekilde anlattık. Grafik arayüzünden bahsettim, ancak tekrar hatırlatayım, bu eğitimde grafik arayüzüne odaklanmayacağız. Evet, grafik arayüzünü kullanacağız, ancak özellikle ele almayacağız. Yani, ofis yazılımlarından, internet tarayıcılarının çalışma şekline, arka plan resminin nasıl değiştirileceğinden veya sesin nasıl kısılacağından bahsetmeyeceğiz. Çünkü bunların hepsini zaten kendi başınıza kolaylıkla keşfedip kullanabilirsiniz. Ayrıca, bahsedecek olsak bile, bu tür grafiksel açıklamalar, kullandığınız dağıtıma ve masaüstü ortamına bağlı olarak çeşitlilik gösterdiği için herkese hitap

edebilecek bir grafik arayüz tanıtımı yapmak mümkün değil. Çünkü Linux sistemi grafik arayüzü sınırsız bir şekilde özelleştirilebiliyor. Bu özelleştirmeler nedeniyle, dağıtımların ve dağıtım sürümlerinin kendine özgü grafik arayüzleri bulunuyor. Dolayısıyla, artık odak noktamız olan komut satırı arayüzünden bahsederek devam etmek en mantıklısı.

Komut Satırı Arayüzü

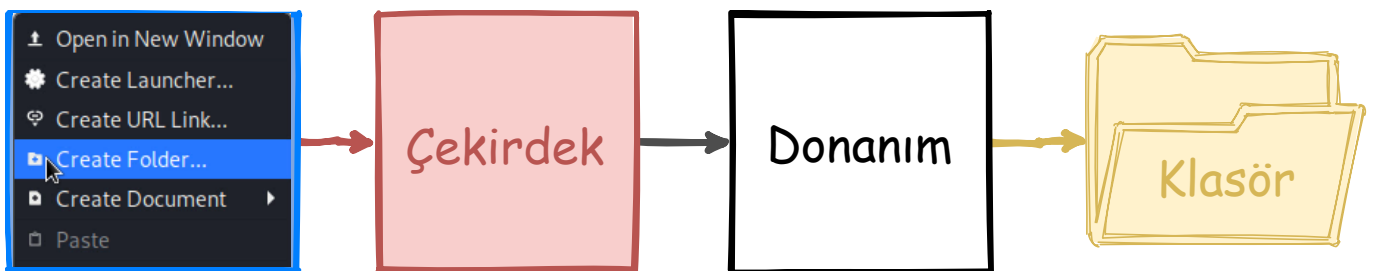
Komut satırı arayüzü, grafiksel arayüze sahip olmayan araçların çalıştırıldığı yazı tabanlı bir çalışma ortamıdır. Bu ortamda grafiksel arayüzle çalışan araçları kullanamazsınız çünkü bu ortam grafiksel arayüzü sunabilecek yapıda tasarlanmamıştır. Komut satırı arayüzünde çalışan araçlar, grafiksel bir etkileşime ihtiyaç duymadan klavye üzerinden yönetilebilen araçlardır. Bu ortamda herhangi bir grafiksel etkileşim olmadığı için araçların bulunup çalıştırılması, araçlar arasında geçiş veya ortak kullanım gibi işler için kabuk olarak isimlendirilen araç bizlere yardımcı oluyor. Nasıl ki grafiksel arayüze sahip uygulamalar masaüstü ortamı(pencereler, butonlar vs..) gibi çevre birimleri sayesinde kolay kullanılabilir oluyorsa, kabuk da komut satırı üzerinden çalıştırılan araçları organize şekilde kullanabilmemiz için bize gereken ortamı sağlıyor. Yani kısacası söz konusu komut satırı arayüzünde çalışmak olduğunda "kabuk" dediğimiz yapı bizim yegane yardımcımızdır. Grafiksel arayüzde masaüstü ortamı neyse komut satırı arayüzünde de kabuk odur.

Örneğin hepimizin bildiği gibi grafiksel arayüzdeki bir uygulamayı çalıştırmak için uygulamanın simgesine tıklamamız gerekir. Komut satırı arayüzünde ise bir aracı çalıştırmak için kabuğa ilgili aracın ismini yazarız. Her iki ortam da bize sistem üzerindeki araçları, bulma çalıştırma ve gerektiğinde sonlandırma gibi konularda yani kısacası "sistemin ve içindeki araçların yönetimi" konusunda yardımcı oluyorlar.

Şimdi çok basit bir örnek üzerinden sistem yönetiminin genel işleyişe bakalım. Örneğin ben yeni bir klasör oluşturmak istersem önümde iki temel seçenek bulunuyor. Bunlardan ilki grafiksel arayüze sahip olan dosya yöneticisini kullanmak, diğeri ise "kabuk" olarak ifade edilen sistem yönetim aracına komut vererek klasörün oluşturulmasını sağlamak.

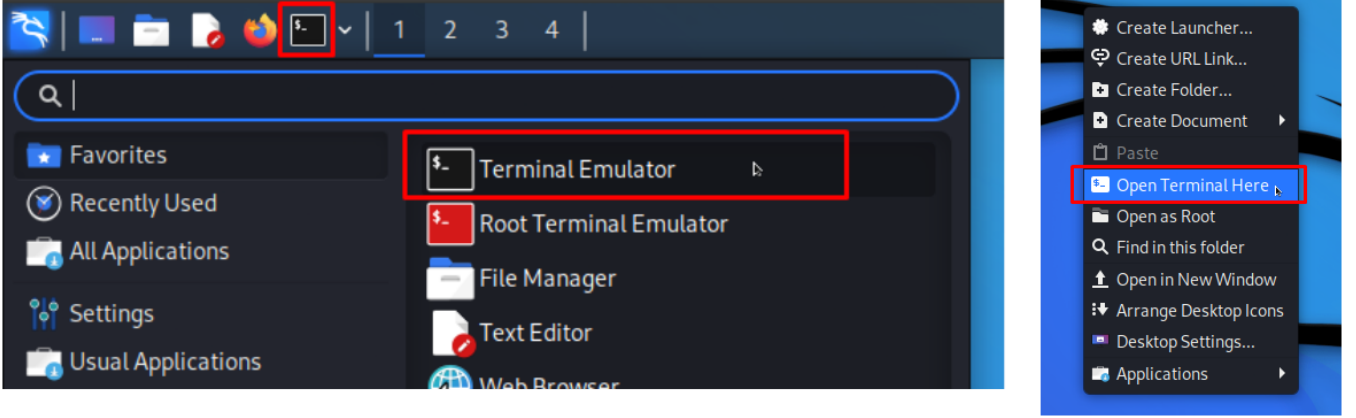
Eğer grafiksel arayüzü kullanarak yeni bir klasör oluşturmak istersem, bir dosya yöneticisi açıp, sağ tıklayıp "yeni klasör oluştur" seçeneğine tıklamam yeterli. Peki ama biz bu işlemleri yapınca işletim sistemi aslında bizim bir klasör oluşturmak istediğimiz nasıl anlıyor ?

Biz yeni bir klasör oluşturmak üzere grafiksel arayüze sahip olan dosya yöneticisi aracını kullandığımızda, aslında bu araç çekirdek ile iletişime geçip bizim gerçekleştirmek istediğimiz işlemi çekirdeğe ifade ediyor. Çekirdek, bu işlem için gereken tüm şartlar müsaitse sisteme bağlı bulunan donanıma bu işin nasıl yapılacağını izah ediyor. Ve en nihayetinde gerekli donanımlar çalıştırılarak bizim klasörümüz oluşturulmuş oluyor. Bakın bu diyagram bu işlem akışının en yalın soyutlamasıdır.

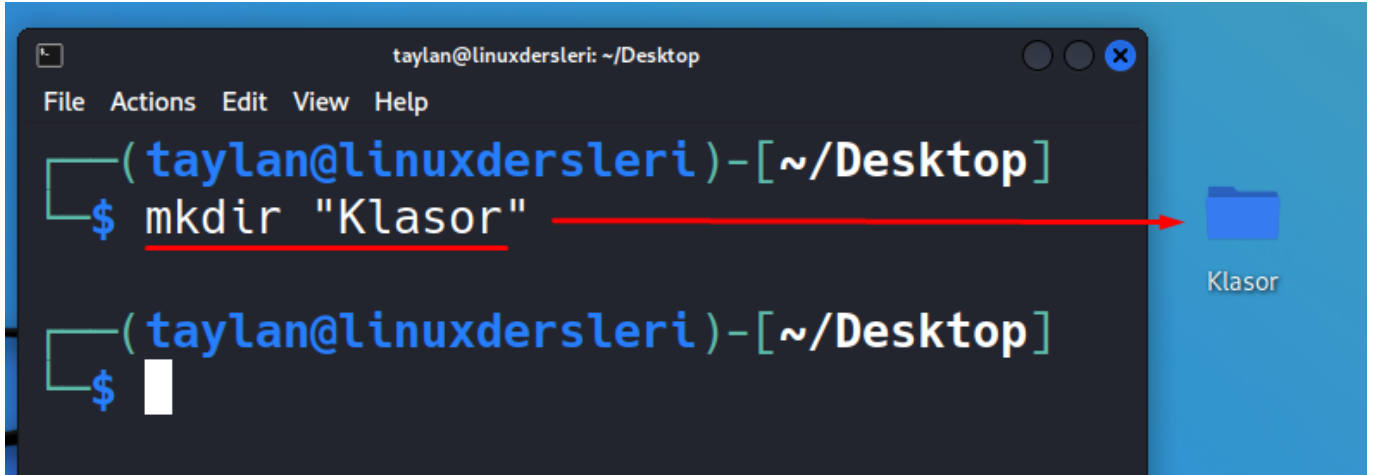


Yani temelde biz klasör oluştururken bu yapılar arkaplanda bizim istediğimizi yerine getirmek için uyum içinde çalışıp görevi yerine getiriyorlar.

Grafiksel arayüze sahip olmayan komut satırı arayüzlü araçlar için de aslında benzer işleyiş geçerli. Bu durumu teyit etmek üzere komut satırı arayüzü ile klasör oluşturulma akışına da göz atabiliriz. Komut girebilmek için öncelikle komut satırı aracını açmamız gerekiyor. Komut satırı aracını yani terminal veya konsol olarak geçen aracı grafiksel arayüzden menüleri kurcalayarak bulup açabilirsiniz. Ayrıca pek çok dağıtımda sağ tıkladığınızda, burada bir konsol başlat gibi bir seçenek de oluyor.

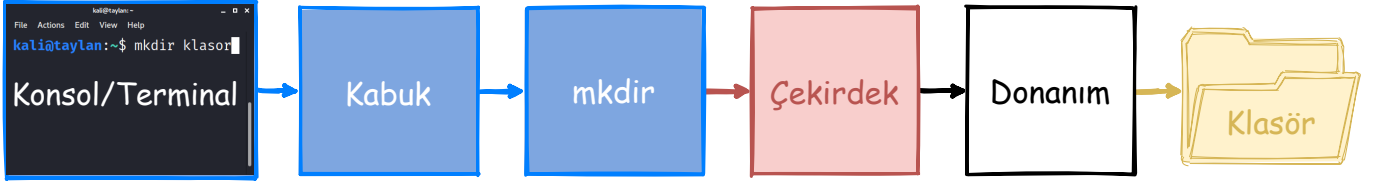


Konsol aracını açtıysak şimdi klasörü oluşturmak üzere `mkdir Klasor` şeklinde komutumuzu girelim.



Bakın belirttiğim isimdeki klasörün oluşturulduğunu grafiksel arayüz sayesinde görebiliyorum. Yani komut satırı üzerinden komut girerek de grafiksel arayüzdeki klasör oluşturma işleminin aynısını gerçekleştirebildim. Peki ama işletim sistemi, girdiğim `mkdir Klasor` kutuyla aslında benim bir klasör oluşturmak istediğimi nasıl anladı ?

Burada grafiksel arayüze gerek kalmadan klasör oluşturmamızı sağlayan araç "`mkdir`" aracıdır. Bu aracı kullanmak istediğimizi sisteme belirtmek için de klavyemizdeki girdileri okuyan "**konsol**" ya da "**terminal**" olarak geçen aracı kullanmamız gerekiyor. Buradaki konsol aracının tek görevi klavyemizdeki girdileri alıp "**kabuk**" olarak geçen yapıya aktarmaktır. Kabuk denilen yapı bizim yazılı şekilde ifade ettiğimiz komutu anlamlandırıp, "`mkdir`" aracını bulup "`Klasor`" argümanı ile çalıştırılmasını sağlıyor. Bu sayede çalıştırılan "`mkdir`" aracı, "`Klasor`" isimli klasör oluşturmak istediğini çekirdeğe izah edebiliyor. Çekirdek de donanıma gereken işi yaptırabiliyor. Neticede yalnızca yazılı emirler ile istediğimiz işlemin gerçekleştirilmesini kolaylıkla sağlamış oluyoruz.



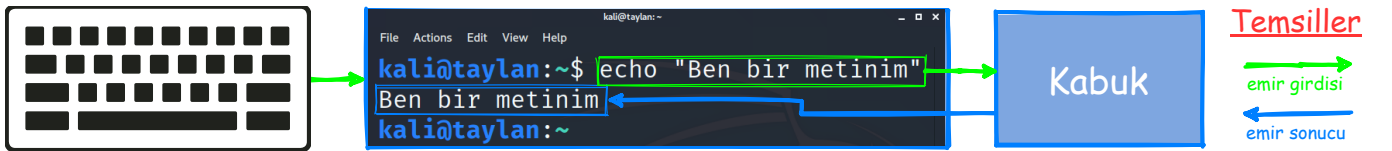
İki örnek üzerinden de görebildiğimiz gibi en nihayetinde bizim işletim sistemine derdimizi anlatmak için bir iletişim yöntemine ihtiyacımız var. Bu iletişim yöntemi de ele aldığımız şekilde grafiksel arayüzlü araçları kullanmak ya da kabuğa komutlar vermek olabilir. Hangi yöntemi ne zaman tercih edeceğimiz de tamamen o anlık ihtiyaçlarımızla ilgili.

Grafiksel ve komut satırı arayüzlerini açıkladığımıza göre bence artık sistemin genel çalışma yapısı hakkında iyi bir temel atmış olduk. Yine de özellikle komut satırı arayüzü konusunda hiç bir soru işareti kalmaması adına anlatımlarımıza temel kavramları netleştirerek devam edelim.

Konsol | Terminal | Komut Satırı

Bizler kabuk dediğimiz yapıya emirlerimizi iletmek ve gerektiğinde kabuğun döndürdüğü sonuçları görebilmek için "konsol" ya da "terminal" olarak isimlendirilen "komut satırı" yazılımlarını kullanıyoruz. Yani adı üstünde komut satırı araçları yalnızca bizim klavyemizdeki girdileri satır satır alarak kabuğa iletmekle ve kabuğun döndürdüğü sonuçları göstermekle mükelleftir. **Verilen emirleri komut satırı araçları yorumlamaz, yorumlayan kabuktur.**

Örneğin ben konsola "Merhaba" yazdırmak üzere `echo "Merhaba"` komutunu girdiğimde, aslında konsol aracı yalnızca benim klavyemdeki girdileri alıp okuyor ve ben enter ile komutu onaylarsam kabuk programına aktarıyor. Girdiğim komutu yorumlayan ve ne yapılması gerektiğine karar veren kabuktur. Girmiş olduğum komut kabuk tarafından yorumlandıktan ve çalıştırıldıktan sonra komutun sonucu yine konsola bastırılıyor. Bu sayede klavyemizi kullanarak konsol üzerinden rahatlıkla komutlar verip, komutların neticelerini de yine konsol üzerinden takip edebiliyoruz.



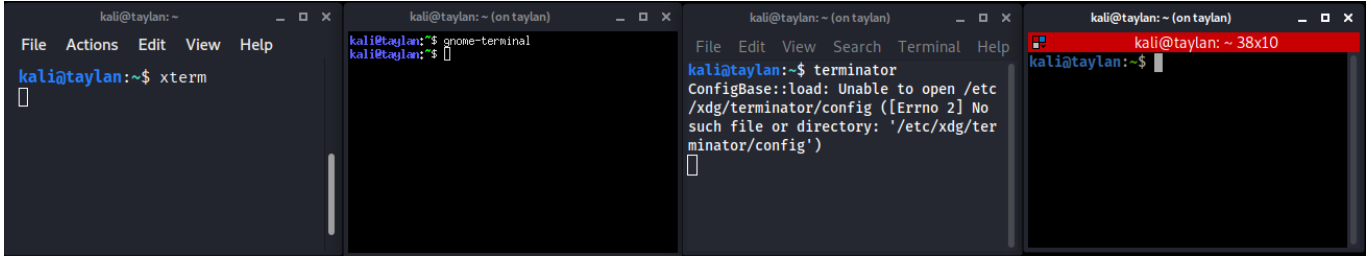
Elbette komut satırı aracı da en nihayetinde bir yazılım olduğu için pek çok farklı alternatifi bulunuyor. Ancak pek çok komut satırı aracı olmasına karşın, aşağı yukarı benzer verimlilikte çalıştığı ve aynı görevi üstlendikleri için hangi aracı kullandığının aslında doğrudan bir önemi yok. Sisteminizde varsayılan olarak yüklü gelen herhangi bir komut satırı yani terminal aracını rahatlıkla kullanabilirsiniz. Fakat dikkatinizi çekmek istediğim küçük bir nüans var.

Komut satırı aslında, grafiksel kullanıcı arayüzünde(graphical user interface GUI) ve komut satırı arayüzünde(command-line interface CLI) olmak üzere iki ayrı ortamda da kullanılabilir. Bu ne demek oluyor hemen izah edelim.

Kabuğa komut girmek için kullandığım bu konsol aracı aslında sizlerin de bildiği gibi grafiksel arayüzde çalışan bir araç. Ben bu aracı grafiksel arayüz sayesinde çalıştırdım ve komutlarımı klavyemi kullanarak kabuğa ilettim kabuk da gereken görevleri yerine getirdi.

Yani özetle grafiksel arayüzde çalışan komut satırı araçları-konsollardan kastım benim şu an kullandığım bu araç gibi olağan şekilde pencere yapıda ve birtakım butonları menüleri olan standart grafiksel arayüzlü

yazılımlardır.



Grafiksel arayüzde çalışan konsol araçlarına; "xterm", "gnome-terminal", "Xfce4 terminal", "Terminator" ve benzeri pek çok alternatif aracı sayabiliriz. Ama neticede hepsinin ortak amacı bizlere grafiksel arayüz ortamında çalışırken bile kabuğa komutlar verebilme imkanı sağlamaktır. Grafiksel arayüzdeyken konsol aracını çalıştırmak için kullanmakta olduğunuz dağıtımın araç çubuğuna, menülerine göz atmanız veya arama çubuğunu kullanmanız yeterli. "Konsol" veya "Terminal" şeklinde arattığınızda zaten yüklü bulunan komut satırı aracına kolayca erişebilirsiniz.

Bu araçlar sayesinde grafiksel arayüz ortamındayken, komut satırı arayüzüne sahip olan araçları da kullanabiliyoruz. Örneğin ben yalnızca komut satırı arayüzünde çalışabilen `mkdir` aracını kullanmak için bu konsol aracını kullandım. Grafiksel arayüzdeki bu konsol aracı olmasaydı, komut satırında çalışan `mkdir` aracını kullanamazdım.

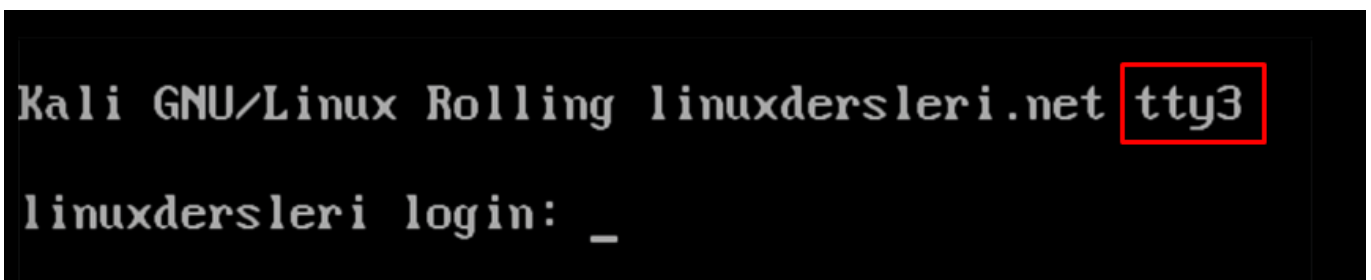
Özetle grafiksel arayüzde çalışırken dahi, konsol yada terminal olarak da bilinen bu araçlar sayesinde kabuk ile etkileşim kurmak için iletişim kanalımız her zaman açık. Zaten bu sayede yalnızca grafiksel arayüze sahip araçlarla sınırlı kalmadan, istediğimiz an komut satırında çalışan araçlar da dahil sistemdeki tüm araçları kolayca yönetebiliyoruz.

Benim daha önce atıfta bulunduğum "komut satırı arayüzü" ise grafiksel arayüze dair hiç bir bileşenin olmadığı ikinci kullanıcı alanıdır. Grafiksel arayüzün bulunmadığı bu komut satırı arayüzünde de yalnızca "tty" olarak geçen sanal konsollar bulunuyor. Bizler de bu tty konsolları üzerinden kabuk ile iletişim kurup komut satırında çalışan her aracı buradan yönetebiliyoruz. Bu ortamda grafiksel arayüzü sağlayabilecek bileşenler bulunmadığı için grafiksel arayüze sahip olan araçları burada kullanamıyoruz.

Bu ortamın nasıl olduğunu daha iyi anlamak için bizzat komut satırı arayüzüne geçiş yapalım.

Grafiksel arayüze sahip olan dağıtımlarda, komut satırı arayüzüne geçiş yapmak için `Ctrl + Alt + F1` `Ctrl + Alt + F2` veya duruma göre diğer `Ctrl + Alt + F3`, `F4`, `F5`, `F6`, `F7` tuşlarından herhangi birisini kullanarak geçiş yapabilirsiniz. Geçiş için hangi kısayolun çalıştığını görmek için tek tek denemeniz gerekiyor. Birden fazla kısayolu denemenizi söylüyorum çünkü geçiş için kullanılması gereken kısayol, kullanmakta olduğunuz dağıtıma ve dağıtım sürümüne göre değişiklik gösterebiliyor. Bu sebeple sırasıyla kısayol tuşlarını deneyebilirsiniz.

Örneğin ben `Ctrl + Alt + F3` tuşlaması ile 3. tty konsoluna geçiş yaptım.



Kaçıncı tty konsolunda olduğunuzu "tty" ardındaki numaraya bakarak öğrenebilirsiniz.. Ben `Ctrl + Alt + F3` tuşlaması yaptığım için 3. tty konsola geçmiş oldum. Bu konsol üzerinden komut girebilmek tabii ki öncelikle konsolda oturum açmamız gerekiyor. Sırasıyla kullanıcı adınızı ve parolanızı yazıp oturum açabilirsiniz. Bu durumu grafiksel arayüzdeki oturum açma ekranı gibi düşünebilirsiniz. Tıpkı grafiksel arayüzde olduğu gibi eğer oturum açmaya yetkimiz yoksa komut satırına da erişemiyoruz. Önce oturum açmamız gerekiyor.

```
Kali GNU/Linux Rolling linuxdersleri.net tty3
linuxdersleri login: taylan
Password:
Linux linuxdersleri.net 5.15.0-kali3-amd64 #1 SMP Debian 5.15.15-2kali1 (2022-01-31) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Jun 24 05:50:59 EDT 2023 on tty3
(taylan@linuxdersleri)~]
$ _
```

Ben kendi hesabımda oturumumu açtım yani artık kendi hesabımın yetkileri dahilinde buradan istediğim gibi komut girebilirim.

Komut girilebildiğini teyit etmek isterseniz `echo "Merhabalar"` şeklinde komut girmeyi deneyebilirsiniz. Sonuçta tıpkı grafiksel arayüzdeki konsolda olduğu gibi bu konsolda da size "Merhaba" çıktısını sonuç olarak döndürecektir.

Ben şimdi yalnızca 3. tty konsoluna geçiş yaptım ancak normalde kısayollar ile 7 tty konsolu üzerinde gezinebilme imkanımız var. Örneğin `Ctrl + Alt + F5` ile 5. tty konsoluna kolayca geçiş yapabiliyoruz. Ancak genellikle grafiksel arayüze sahip bir dağıtım kullanıyorsanız, bu tty konsollarından bir veya ikisi mutlaka grafiksel arayüz için ayrılmıştır. Yani örneğin sizin kullandığınız dağıtımda 1. tty konsolu grafiksel arayüz için ayrıldıysa `Ctrl + Alt + F1` komutu ile tekrar grafiksel arayüze dönebilirsiniz.

Neticede bu şekilde `Ctrl + Alt` ve `F1 - F7` kısayolları ile hem birden fazla komut satırı arayüzüne yani tty konsoluna hem de mevcutsa grafiksel arayüze geçiş yapabilirsiniz. Neticede komut satırı arayüzü dediğimiz kullanıcı bölümü burası yani komut satırı arayüzü böyle gözüküyor.

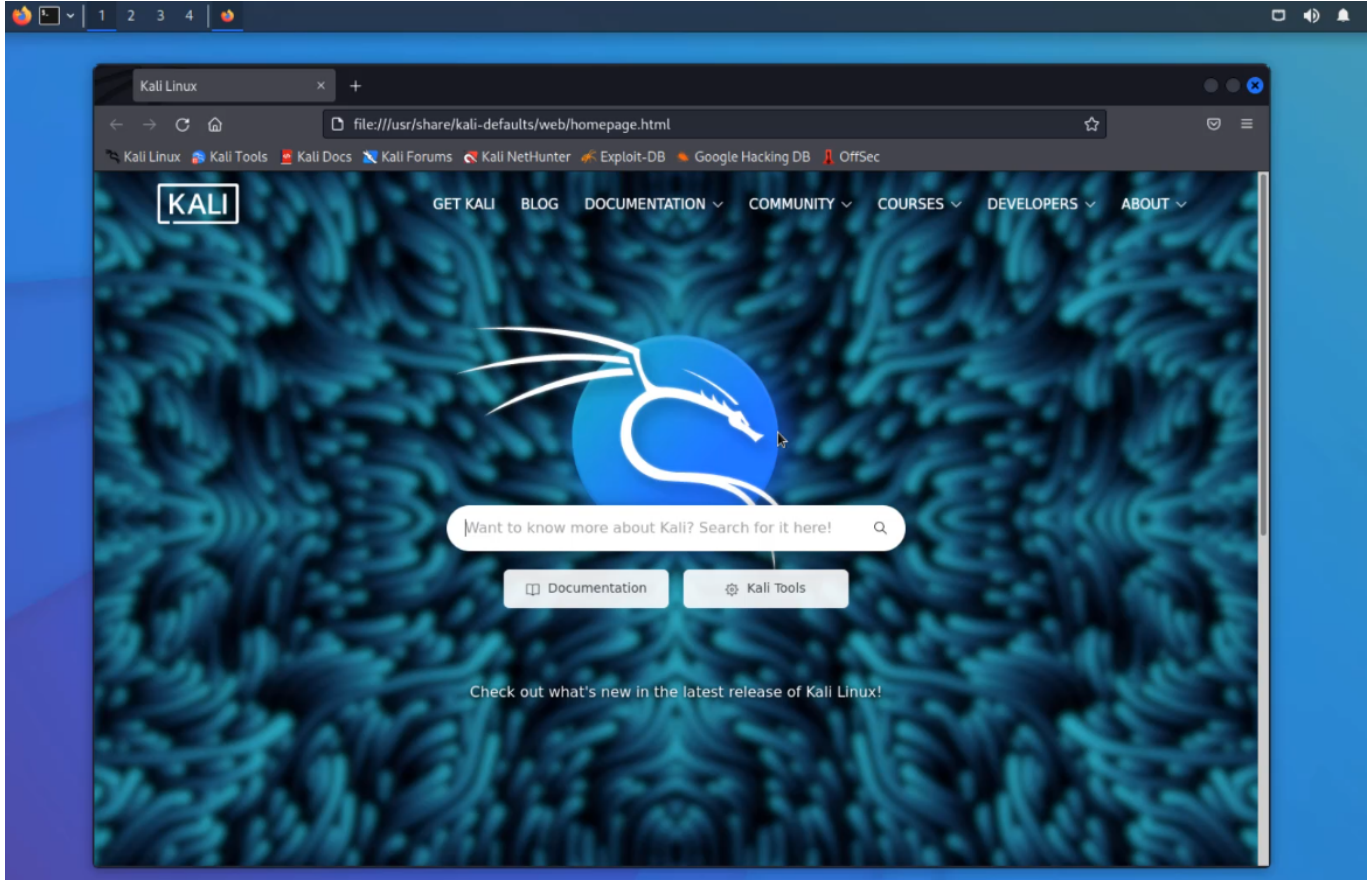
Komut Satırı Arayüzünde Çalışmak

Tahmin edebileceğiniz gibi grafiksel arayüze sahip olan uygulamaları yalnızca grafiksel kullanıcı ortamında açabiliyoruz. Komut satırı arayüzü ile çalışan araçları ise daha önce bahsettiğimiz grafiksel arayüzdeki konsol araçları veya tty konsolları sayesinde her iki ortamda da çalıştırmamız mümkün oluyor. Daha net anlaşılması için bu durumu örnek üzerinden açıklamaya çalışalım.

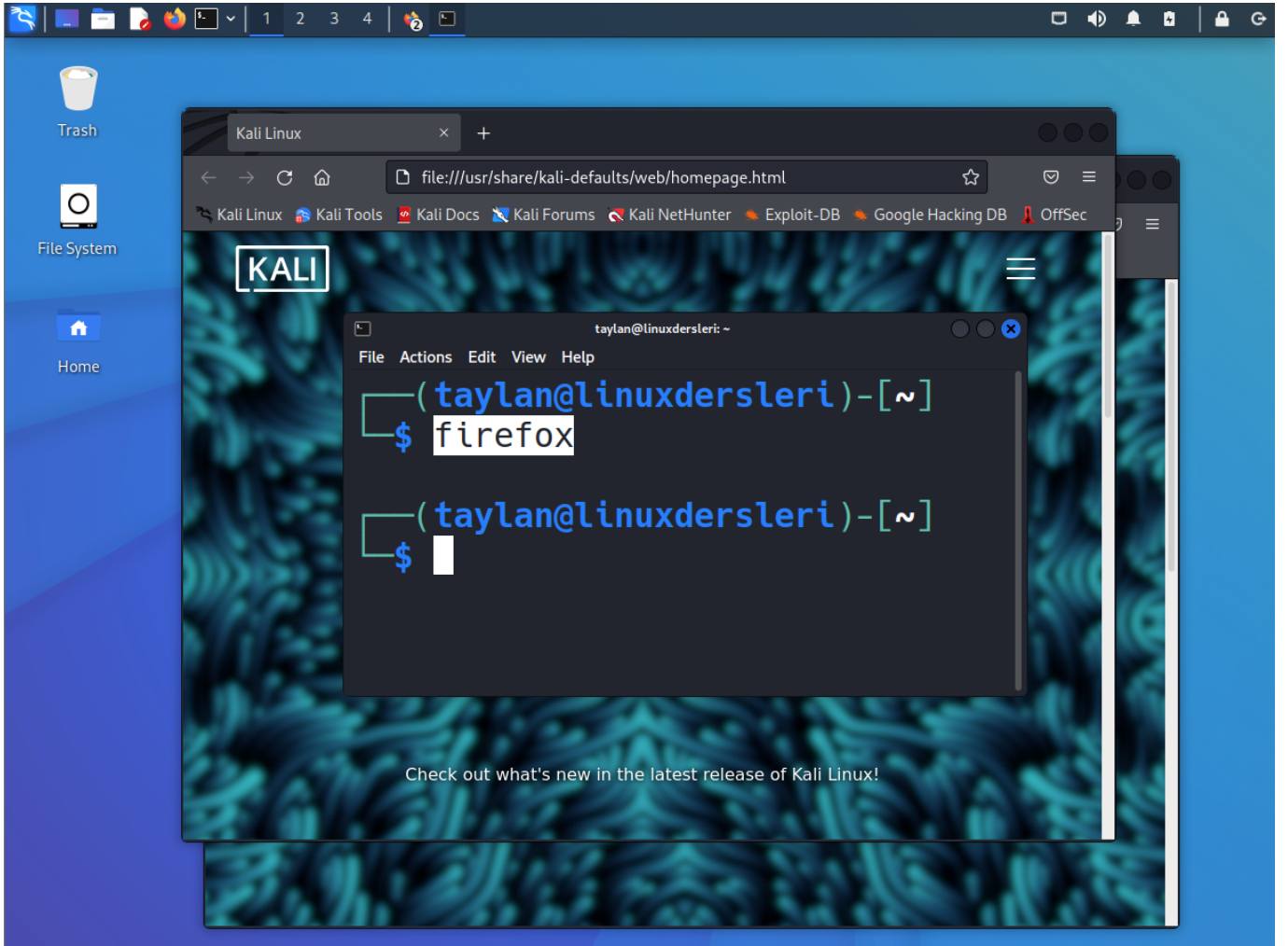
Örnek vermek için farklı arayüzlerde çalışan iki farklı aracı ele alabiliriz. Birincisi grafiksel arayüzde çalışan "**firefox**" isimli tarayıcı, diğeri ise komut satırı üzerinde çalışan "**nano**" aracı.

Örneğin eğer sistemde grafiksel arayüzde çalışan bu firefox isimli araç yüklü ise ben grafiksel arayüzdeyken faremi ve menüleri kullanarak firefox aracını bulup üzerine tıklamak suretiyle çalıştırabilirim.

i Not: Eğer sizin kullanmakta olduğunuz dağıtımda firefox aracı yüklü değilse, beni takip ederken grafiksel arayüzde çalışabilecek herhangi bir aracı da test için kullanabilirsiniz. Burada önemli olan tek şey grafiksel arayüze çalışan herhangi bir aracı çalıştırmak. İsminin ne olduğu önemli değil. Ben test ederken firefox aracını kullanacağım.

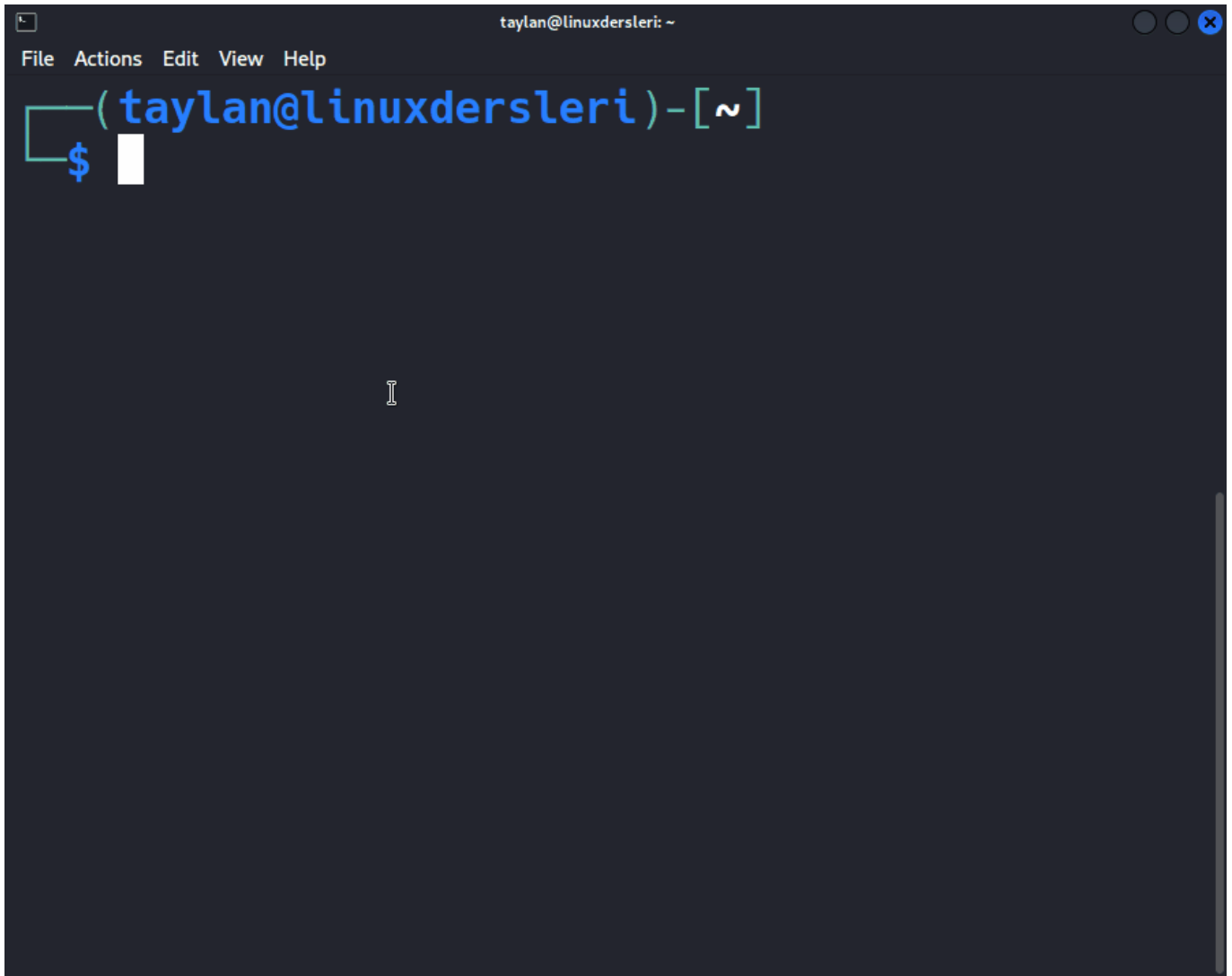


Firefox aracı benim sistemimde yüklü olduğu için bulabildim ve üzerine tıkladığımda açıldı. Hatta bu aracı bulup çalıştırmak için aslında bu menüyü ya da arama çubuğunu kullanmak zorunda da değilim. Grafiksel arayüzdeki menülere ek olarak, terminale `firefox` yazarak kabuğun benim için bu aracı bulup mevcut grafiksel ortamda açmasını da sağlayabilirim.



Bakın grafiksel arayüze sahip olan aracımı komut satırına komut girerek de çalıştırabildim. Neticede ben şu an grafiksel arayüzde çalıştığım için, grafiksel arayüze sahip olan aracımı her iki şekilde de çağırıp çalıştırabildim. Çünkü şu an grafiksel arayüzün çalışabileceği uygun ortama sahibim.

Testi devam ettirmek için yalnızca komut satırında çalışan **nano** isimli metin editörü aracını da ele alabiliriz. Örneğin **Nano** aracı yalnızca komut satırı üzerinden çalıştığı için bu aracı herhangi bir konsol uygulaması üzerinden çalıştırabilirim. Denemek için hemen konsola nano komutunu girelim.



Bakın grafiksel arayüzdeki konsol aracına **nano** komutunu girdiğimde **nano** aracı konsolda açılmış oldu. Artık bu konsol üzerinden nano aracını kullanıp, metin belgisi oluşturabilir ya da var olanları düzenleyebilirim. Yani konsol yardımıyla grafiksel arayüzde çalışıyor olmama rağmen komut satırı üzerinden çalışabilen nano uygulamasını da kullanabiliyorum gördüğünüz gibi.

Şimdi buradaki örneklerin aynılarını bu kez komut satırı arayüzüne geçiş yapıp tekrar deneyebiliriz. Ben **Ctrl + Alt + F5** ile 5. tty konsoluna geçiş yapıyorum. Siz istediğiniz bir tty konsoluna geçiş yapabilirsiniz.

İlk olarak grafiksel arayüze sahip olan firefox aracını çalıştırmak için **firefox** komutunu girmeyi deneyebiliriz.

```
Kali GNU/Linux Rolling linuxdersleri.net tty5

linuxdersleri login: taylan
Password:
Linux linuxdersleri.net 5.15.0-kali3-amd64 #1 SMP Debian 5.15.15-2kali1 (2022-01-31) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 28 01:05:57 EDT 2023 on tty3
(taylan@linuxdersleri)~]
$ firefox
Error: no DISPLAY environment variable specified

(taylan@linuxdersleri)~]
$ _
```

Bakın, firefox aracı bulunmasına rağmen bu aracı çalıştıracak grafiksel arayüz ortamı bulunmadığı için hata aldık.

Şimdi bir de `nano` komutunu girip nano aracını çalıştırmayı deneyelim.

```
Kali GNU/Linux Rolling linuxdersleri.net tty5

linuxdersleri login: taylan
Password:
Linux linuxdersleri.net 5.15.0-kali3-amd64 #1 SMP Debian 5.15.15-2kali1 (2022-01-31) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 28 01:05:57 EDT 2023 on tty3
(taylan@linuxdersleri)~]
$ firefox
Error: no DISPLAY environment variable specified

(taylan@linuxdersleri)~]
$ _
```

Bakın nano aracı komut satırı arayüzünde çalışan bir araç olduğu için burada da sorunsuzca açıldı. İşte bu şekilde komut satırı arayüz ve grafiksel arayüz arasındaki farkı bizzat uygulamalı olarak test etmiş olduk.

Grafiksel arayüzlü araçları yalnızca grafiksel arayüz ortamında kullanabiliyorken, komut satırı arayüzünde çalışan araçları, grafiksel arayüzde çalışan konsol araçları yardımıyla hem grafiksel arayüzden hem de komut satırı arayüzündeki tty konsolları üzerinden kullanabiliyoruz.

Bu arada eğer nano aracını kapatmak istiyorsanız `Ctrl + x` tuşlaması ile kapatıp, tekrar komut satırına dönebilirsiniz.

Neticede uyguladığımız örneklerle birlikte hem grafiksel arayüzde hem de komut satırı arayüzünde, terminal araçları sayesinde sisteme yazılı emirler verebileceğimizi bizzat deneyimlemiş olduk.

İki farklı kullanıcı arayüzüne sahip olsak da bizler eğitim boyunca grafiksel arayüzdeki konsol araçları yardımıyla sistemi komutlar vasıtasıyla nasıl yönetebileceğimize odaklanacağız.

Aslında doğrudan grafiksel arayüzün bulunmadığı komut satırı arayüzü üzerinden de ilerleyebilirdik fakat bu yeni başlayan kullanıcıların ekstra zorluk yaşamasına sebep olabileceği için bence ilk aşama için pek doğru bir tercih olmaz. Ayrıca tıpkı klasör oluşturma örneğimizde olduğu gibi, girilen komutların etkilerini grafiksel arayüzden takip edebilme imkanı, öğrenilen kavramların görsel olarak daha da netleşmesini sağlayacağı için grafiksel arayüzdeki konsollar üzerinden çalışmak bence yeni başlayan kullanıcılar için daha doğru bir tercih.

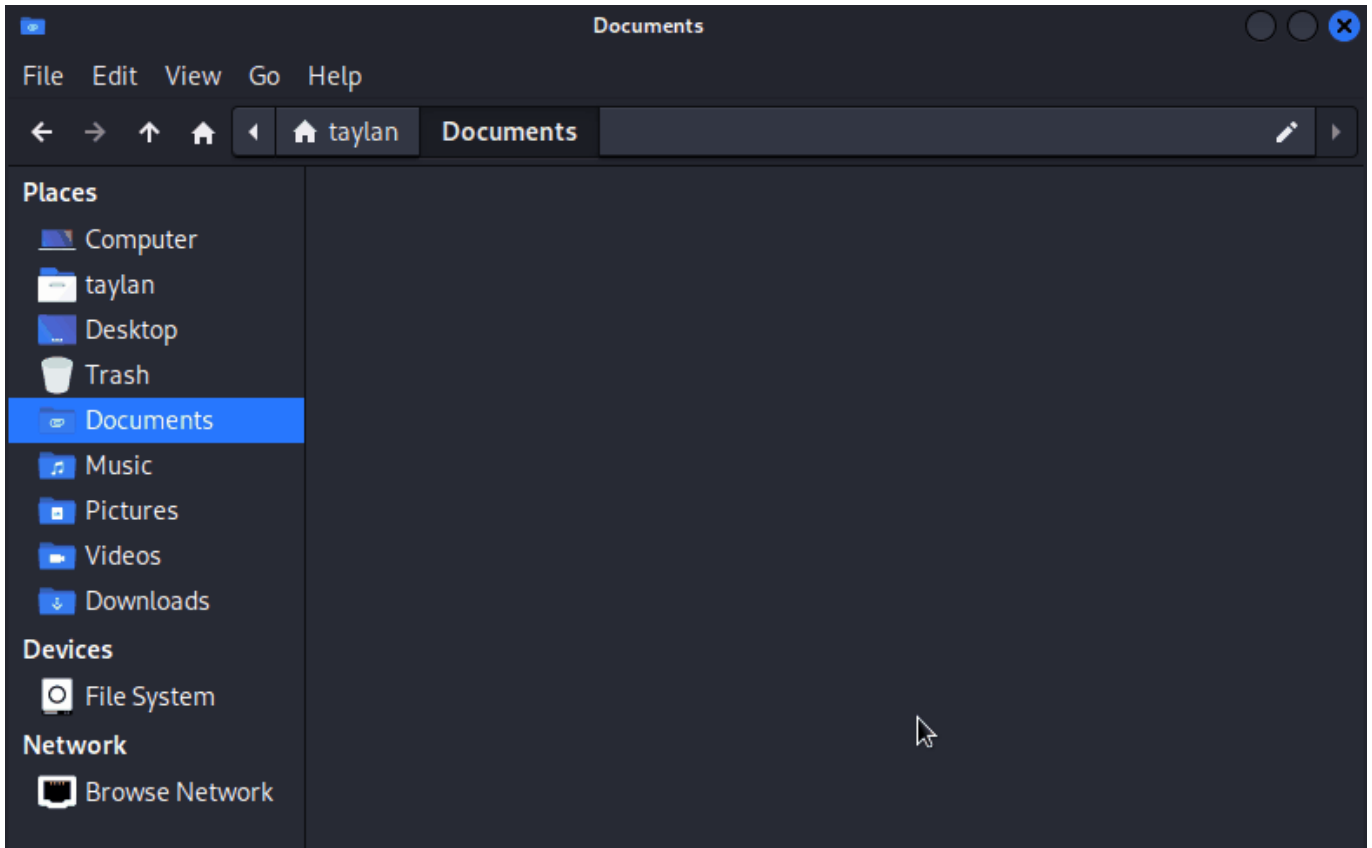
Yine de daha önce söylediğim şekilde grafiksel arayüzdeki konsol araçlarını kullanacak olmamız sizi aldatmasın. Özellikle grafiksel arayüzün nasıl kullanıldığı gibi detaylara değinmeyeceğiz. Bu noktada muhtemelen "neden kolayca kullanılabilir grafiksel arayüz araçları varken yalnızca yazılı komutlar ile sistemi yönetmek isteyelim ki ?" diye düşünmüş olabilirsiniz.

İlk bakışta her ne kadar öyle gözükmesi de aslında komut satırı kullanımı yani sistemi komutlarla yönetmek aslında çok daha verimlidir. Grafiksel arayüzler sistem kaynaklarını fazlaca kullanılıyor ve yalnızca grafiksel arayüzün sunduğu sınırlı özellikler dahilinde işlerimizi yerine getirebiliyoruz. Komut satırı kullanımında ise doğrudan yapılacak işlem tüm detaylarıyla belirtilebildiği ve aynı anda birden fazla aracın bir arada kullanılabilmesi mümkün olduğu için çok daha esnek ve verimli bir çalışma ortamına sahip oluyoruz.

Hatta konsola gireceğimiz tüm komutları gerektiğinde bir dosyaya(betik dosyası olarak geçer) kaydedip daha sonra istediğimiz bir zamanda otomatik olarak çalıştırılmasını da sağlayabiliyoruz. Yani komutların kullanımı, biz bilgisayarın başında değilken dahi sistemin yönetilebilmesine olanak tanıyor. Kısacası yazılı emirler ile sistemi yönetmek, grafiksel arayüze oranla bizlere çok daha esnek ve verimli yönetim imkanı sağlıyor.

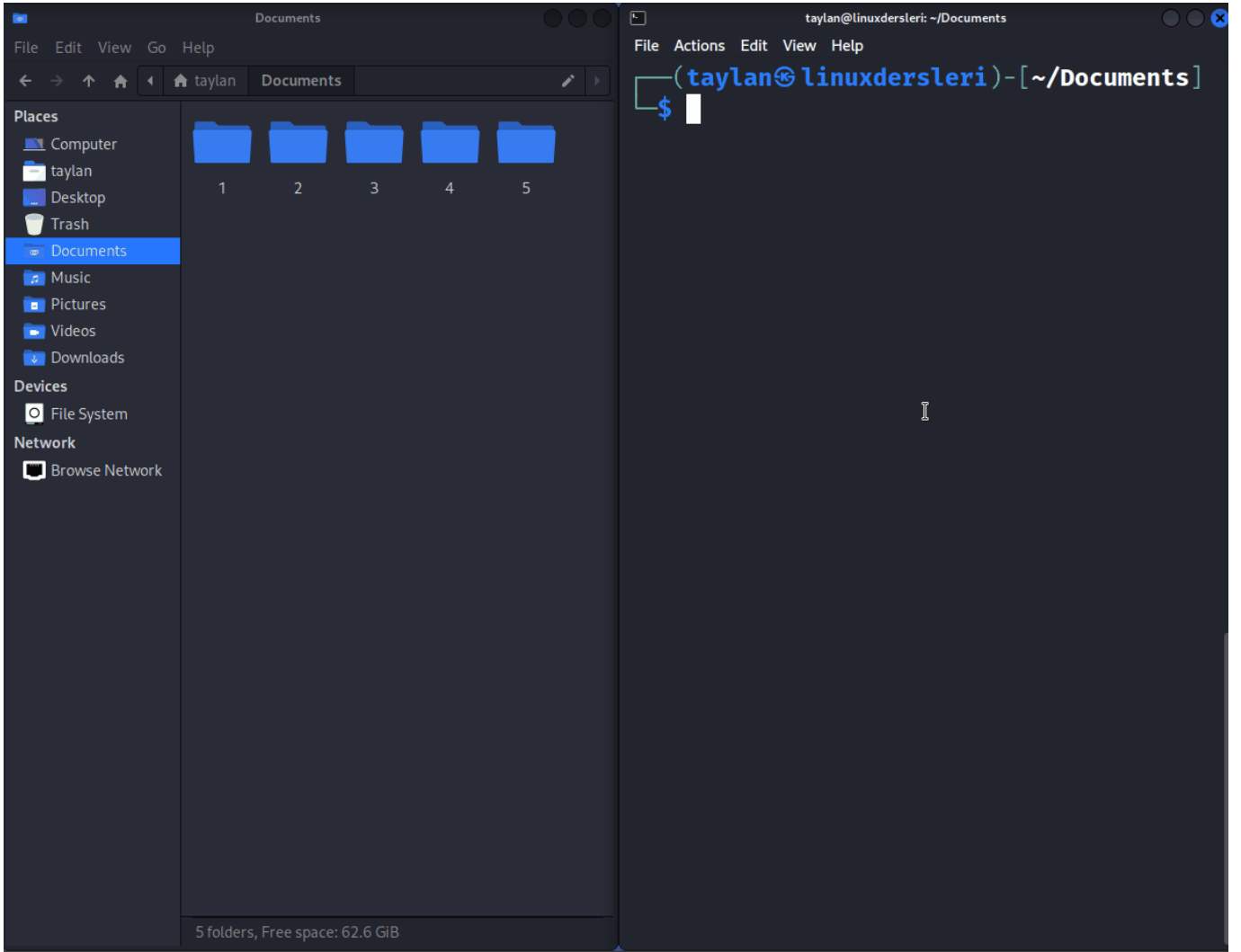
Verimlilik konusuna henüz bana katılmıyor olabilirsiniz ancak merak etmeyin çünkü eğitimin devamında tamamen hak veriyor olacaksınız. Hatta eğitime devam etmeden bu esnekliği kanıtlamak için sizinle çok basit bir örnek yapabiliriz.

Örneğin bulunduğunuz konumda grafiksel arayüzü kullanarak 1 den 100'e kadar isimlendirilmiş klasörler oluşturmanız ne kadar sürer diye düşünün lütfen ? Tahminimce bu klasör oluşturma işi en az 3-4 dakikayı bulabilir.



Öte yandan aynı işi komut satırından 5 saniyede gerçekleştirebilirsiniz. Hadi hemen deneyelim.

Komutun sonucunu rahat gözlemleyebilmek için dosya yöneticisini ve terminali yan yana alalım ve konsola `mkdir {1..100}` komutunu girelim.



Bakın anında 1 den 100 e kadar isimlendirilmiş olan klasörler tam olarak bu dizinde tek bir komutla oluşturuldu. Bence bu basit örnek komut kullanımının ne kadar verimli olabileceğini gözler önüne seriyor. Tabii ki bu örnek komut kullanımının verimini açıklama konusunda oldukça basit bir örnek. Ancak merak etmeyin ileride kendi başına basit bir işi yapan, birbirinden farklı pek çok aracı bir arada kullanarak çok daha kompleks sorunlara nasıl çözüm getirebileceğinizi bizzat deneyimlemiş olacaksınız. Buradaki "kendi başına basit işleve sahip araçları bir arada kullanmak" ifadesi de aslında oldukça önem arz ediyor ancak bunun üzerinde daha sonra duruyor olacağız. Şimdi kafanızı karıştırmak istemiyorum.

Sanırım artık basit örneğimiz sayesinde, yazılı şekilde komutlar girmenin grafiksel arayüzden avantajlı olabileceğini konusunda hemfikir olduk. Artık komutlar ile haşır neşir olacaksak, girdiğimiz komutları anlayan ve çalıştırılmasına aracılık eden kabuk programını da daha yakından tanısak hiç fena olmaz. Zira Linux sistemini etkili şekilde yönetebilmeniz tamamen kabuğu ne kadar etkili şekilde kullanabildiğinizle ilgili. Anlatımlarımıza kabuktan bahsederek devam edeceğiz ancak kabuğu açıklamadan önce ilerleyişimiz hakkında kısa bir hatırlatmada bulunmak istiyorum.

Biliyorum pek çok kavramı birden fazla kez tekrar etmem pek çoğunuz için hoş bir durum değil. Ancak tüm kavramların birbiri ile yakından ilişkili olması ve asıl hedef kitlenin sıfırdan öğrenmek isteyen kişiler olması sebebiyle yeni öğrendiğimiz bilgiler ışığında geriye dönük hatırlatmalar yapmak zorundayım. Eğitime devam ettikçe bu durumun, öğrenilen tüm kavramların temellendirilmiş olması açısından son derece önemli olduğunu zaten sizler de fark etmiş olacaksınız. Şimdiye kadar yaptığımız gibi tüm eğitim boyunca mümkün oldukça sebep sonuç ilişkisine odaklanmaya çalışmamızın sonucu olarak birtakım sorular sorup bunlara yanıt arıyor olacağız. Bu doğrultuda bazen öğrendiğimiz bilgileri tekrar edeceğiz. Çünkü yalnızca bu sayede

"öğrenmeyi öğrenmiş" olur ve gerektiğinde kendi başımızın çaresine bakabiliriz. Zaten eğitimin ele alınış biçimi nedeniyle "temel" olduğuna pek çok kez vurgu yapmıştım. Beklentilerinizi de bu doğrultuda tutarsanız, eğitimden alacağınız faydayı maksimize edebilirsiniz.

Kabuk

Grafiksel arayüze sahip uygulamaları hariç tuttuğumuzda, kabuk sistem yönetimi için olmazsa olmazdır.

Çünkü kabuk bize sistem üzerindeki tüm araçlar için ortak çalışma ortamı sağlıyor. Biz kullanmak istediğimiz aracın ismini ve nasıl kullanmak istediğimizi kabuğa belirtiyoruz, kabuk da ilgili aracı sistem üzerinde bulup uygun şekilde çalıştırılmasını sağlıyor. Ayrıca kabuk kendi bünyesinde de pek çok kullanışlı aracı barındırıyor. Hatta daha önce de kısaca bahsettiğim şekilde rutin işlerinizi otomatikleştirmek üzere komutlarımızı bir dosyaya toplayıp çalıştırmamız yani aslında kabuğumuzu programlamamız da mümkün.

Kısacası sisteme yazılı şekilde derdimizi anlatmak istediğimizde muhatabımız "**kabuk**"tur.

Hatırlıyorsanız, önceki anlatımlarımızda hangi "terminal/konsol" aracını kullandığımızın bir önemi olmadığını söylemiştik. Buna karşın kullandığınız kabuk yazılımının hangisi olduğu çok önemli bir detaydır. Çünkü Bash, zsh, csh, sh ve fish gibi pek çok farklı kabuk yazılımı bulunuyor. Kabuğun birden fazla alternatifinin olması belki sizi şaşırtmış olabilir. Ancak en nihayetinde kabuk dediğimiz yapı da aslında kullanıcı alanında çalışan bir yazılımdır. Dolayısıyla her yazılım gibi elbette kabuğun da pek çok alternatifi bulunuyor. Her ne kadar biz bu eğitimde en yaygın kullanıma sahip olan bash kabuğuna odaklanacak olsak da alternatif kabuklar ile karşılaşmanız da olasıdır.

Bash olarak bahsi geçen kabuk, atası olan sh kabuğunun, csh gibi farklı kabuklardan alınan iyi özellikler dahilinde geliştirilmesiyle ortaya çıkan bir kabuktur. Uzun süredir kararlı şekilde geliştirildiği için dağıtımların çok büyük bir kısmında varsayılan olarak bash kabuğu kullanılıyor. Ben de yaygınlığı dolayısıyla anlatımlar sırasında hep bash kabuğunu kullanıyor olacağım.

Mevcut sistemimizdeki varsayılan olarak kullanılan kabuğun hangisini olduğunu öğrenerek kabuğu daha yakından tanımaya başlayabiliriz.

Varsayılan kabuk programını öğrenmek için konsola `echo $SHELL` komutunu girmeniz yeterli. Bu komutta yer alan `echo` ifadesi varsayılan kabuğun değerini tutan `SHELL` değişkenini konsola bastırmanızı sağlıyor.

```
~ → echo $SHELL
/usr/bin/zsh
```

Bakın ben çıktısı olarak `/usr/bin/zsh` ifadesini aldım. Aldığım bu çıktı benim mevcut kullanıcı hesabımdaki kabuğun **zsh** kabuğu olduğunu açıkça belirtiyor. Normalde pek çok köklü dağıtım varsayılan olarak bash kabuğunu kullanıyor olsa da benim şu an kullanmakta olduğum Kali Linux dağıtımı varsayılan olarak zsh kabuğunu kullanıyor. Fakat ben zsh kabuğunu üzerinden ilerlemek istemiyorum çünkü bash kabuğu hem bireysel hem de profesyonel anlamda çok daha yaygın kullanıma sahip olduğu için bash kabuğunu öğrenmek çok daha mantıklı bir yaklaşım.

Siz de `echo $SHELL` çıktısında hangi kabuğun ismini aldıysanız, şu anda o kabuğa komutlar veriyorsunuz. Eğer sizin hesabınızdaki varsayılan kabuk benimki gibi zsh veya farklı bir kabuksa hiç bir problem yok. Çünkü eğer isterseniz varsayılan kabuğunuzu bash olarak değiştirmeniz mümkün. Nitekim ben de zsh kabuğunu

bash ile değiştireceğim. Sizin kabuğunuz da bash değilse, mevcut eğitimi sorunsuzca takip etmek için sizin de değiştirmeniz gerekiyor. Çünkü biz bu eğitimde bash kabuğunu kullanıyor olacağız.

Zaten Bash kabuk yazılımı yaygın kullanımı sebebiyle istisnai durumlar hariç hemen her dağıtımda yüklü şekilde geliyor, ancak varsayılan kabuk olarak tanımlanmamış olabiliyor. Gerekliğinde bash kabuğunu kullanmak için tek yapmanız gereken sizin kullanıcı hesabınız için varsayılan olarak tanımlamaktır.

Bash Kabuğunu Varsayılan Olarak Ayarlamak

Kabuğu Bash'ten farklı olan kullanıcılar için kabuk değişimini ele alacağız ancak halihazırda kabuğu bash olanlar da dahil herkesin buradaki anlatımları takip etmesini rica ediyorum. Çünkü ileride varsayılan kabuğun farklı olduğu bir sistemle karşılaşabilirsiniz. Yani burada öğreneceğiniz kabuk değiştirme yöntemi mutlaka yeri geldiğinde işinize yarayacak.

Öncelikle mevcut sistemimizde bash kabuk yazılımının **hangi konumda olduğunu** öğrenmek üzere konsola `which bash` komutunu girmemiz gerekiyor.

```
~ → which bash
/usr/bin/bash
```

Bu almış olduğumuz çıktı bash kabuğunun sistem üzerindeki tam konumunu gösteriyor. Bu konumu kopyalayın çünkü birazdan lazım olacak.

Bash kabuğunu kendi kullanıcı hesabımın varsayılan kabuğu olarak tanımlamak için, kabuk bilgisi de dahil sistem üzerindeki tüm kullanıcılara ait çeşitli bilgilerini tutan **/etc/passwd** dosyasını açmam gerekiyor. Bu dosyayı açmak ve düzenlemek için daha önce bahsetmiş olduğumuz nano metin editörünü kullanabiliriz. Bu dosya yalnızca yetkili kişilerce açılıp düzenlenebileceği için komutun başına `sudo` komutunu da eklemeliyiz. Yani `sudo nano /etc/passwd` komutunu girmemiz gerekiyor. Yetki gerektiren bir işlem yaptığımız için bu komutun ardından mevcut kullanıcı hesabımızın parolasını girip enter ile onaylamamız gerek.

```
(taylan@linuxdersleri)-[~]
└─$ sudo nano /etc/passwd
[sudo] password for taylan: BURAYA PAROLANIZI GİRMENİZ GEREK
```

Şimdi açılmış olan passwd dosyasında kendi kullanıcı adımızın geçtiği satırı bulmamız gerek. Dosya içeriğinde gezinmek için klavyemizdeki aşağı yukarı yön tuşlarını kullanabiliriz. Benim kullanıcı adım "taylan" olduğu için "taylan" olan satırı buluyorum. Siz de hangi kullanıcının kabuğunu değiştirmek istiyorsanız o kullanıcının satırını bulmanız gerekiyor. Yani kendi hesabınızı düzenliyorsanız kendi kullanıcı adınızı içeren satırı buradan bulmanız gerekiyor.

```

GNU nano 6.0 /etc/passwd *
stunnel4:x:121:127::/var/run/stunnel4:/usr/sbin/nologin
nm-openconnect:x:122:128:NetworkManager OpenConnect plugin,,:/var/lib/NetworkManag>
Debian-snmp:x:123:129::/var/lib/snmp:/bin/false
speech-dispatcher:x:124:29:Speech Dispatcher,,:/run/speech-dispatcher:/bin/false
sslm:x:125:130::/nonexistent:/usr/sbin/nologin
postgres:x:126:131:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash
pulse:x:127:132:PulseAudio daemon,,:/run/pulse:/usr/sbin/nologin
saned:x:128:135::/var/lib/saned:/usr/sbin/nologin
inetsim:x:129:137::/var/lib/inetsim:/usr/sbin/nologin
lightdm:x:130:138:Light Display Manager:/var/lib/lightdm:/bin/false
colord:x:131:139:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nolog>
geoclue:x:132:140::/var/lib/geoclue:/usr/sbin/nologin
king-phisher:x:133:141::/var/lib/king-phisher:/usr/sbin/nologin
taylan:x:1000:1000:,,:/home/taylan:/usr/bin/zsh

```

Bakın burada kullanıcı adının geçtiği satırın sonunda gördüğünüz gibi benim kullanıcı hesabım için tanımlı olan kabuk programının dosya konumu belirtilmiş. Benim kabuğum zsh olduğu için bu kabuğun dosya konumu yazıyor ancak sizde hangi kabuk programı varsa burada o kabuk programının dizin adresi olacaktır. Örneğin sizin hesabınız için burada sh ya da fish kabuklarının dosya konumları bulunuyor olabilir. Kabuğu değiştirmek için tek yapmamız gereken, hangi kabuğu kullanacaksınız o kabuğun dosya konumunu tam olarak buraya yazmak.

Ben bash kabuğunu varsayılan kabuğum yapmak için daha önce `which bash` komutu sayesinde öğrenmiş olduğum bash kabuk programının tam dosya konumunu zsh yerine ekliyorum.

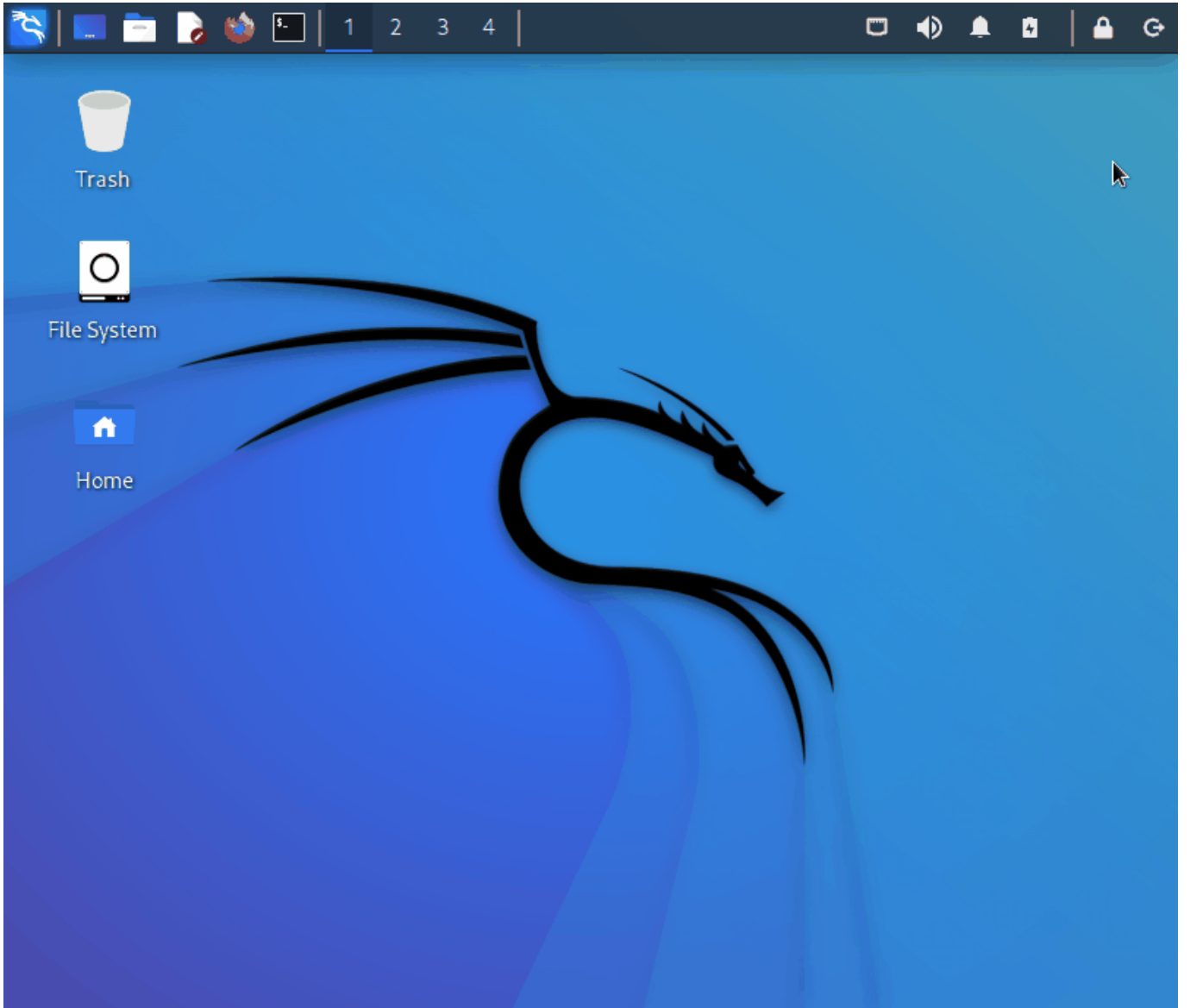
```

king-phisher:x:133:141::/var/lib/king-phisher:/usr/sbin/nologin
taylan:x:1000:1000:,,:/home/taylan:/usr/bin/bash

```

Dosyadaki değişikliği kaydedip nano aracından çıkmak için `Ctrl + X` kısayolunu kullanıp “*değişiklik kaydedilsin mi*” sorusuna da “yes” ile onay vermemiz gerek.

Böylelikle kendi kullanıcı hesabımız için varsayılan kabuğu bash olarak değiştirmiş oluyoruz. Ancak bu değişiklik biz tekrar oturum açtığımızda geçerli olacak. Çünkü bu dosya, biz oturum açarken okunuyor. Örneğin ben “taylan” kullanıcısı olarak giriş yaptığımda, bu dosyadaki “taylan” satırını bulunup varsayılan kabuğun hangi olduğu öğreniliyor. Şimdi yapmış olduğunuz değişikliğin geçerli olabilmesi için oturumumuzu kapatıp tekrar açmanız yeterli. Oturumu nasıl kapatacağınızı bilmiyorsanız, genellikle grafiksel arayüzde sağ üst köşede oturum kapatma seçeneği olur biraz kurcalarsanız bulabilirsiniz. Tekrar oturum açtıktan sonra değişikliğin geçerliliği teyit etmek için tek yapmanız gereken tekrar `echo $SHELL` komutunu girmek.



```
(taylan@linuxdersleri) - [~]  
$ echo $SHELL  
/usr/bin/bash
```

Aldığım bu çıktı, varsayılan kabuk programının benim kullanıcı hesabım için bash olarak değiştiğini kanıtlıyor.

İşte sizler de bu şekilde istediğiniz kullanıcı hesabının varsayılan kabuğunu **/etc/passwd** dosyasından değiştirebilirsiniz. **/etc/passwd** dosyası sisteme erişimi olan tüm kullanıcıların kaydını tutmak için var. Bu dosyadan daha sonra tekrar bahsedeceğiz zaten. Şimdilik kullanıcıların varsayılan kabuğunu bu dosya üzerinden değiştirebildiğimizi bilmemiz yeterli.

Ayrıca kabukla ilgili anlatımları noktalamadan önce, özellikle neden bash kabuğunu kullandığımızdan da bahsedelim istiyorum. Ancak bash kabuğunu neden tercih ettiğimizi ele almadan önce kabuk denilen yapının daha net anlaşılabilmesi için birkaç kavramdan daha bahsetmemiz gerekiyor.

Kabuğa Girilebilecek Komut Türleri

Temelde bizler kabuğa iki tür komut girebiliyoruz. Bu türler “dahili” ve “harici” olarak gruplanmış olan komutlardır.

Dahili Komutlar(Built-ins)

Dahili komutlar, kabuk programında yerleşik olan araçları çalıştırmak üzere kullanılan komutlardır. Bash üzerinde yer alan tüm dahili komutları görmek için `compgen -b` komutunu kullanabiliriz.

```
(taylan@linuxdersleri) - [~]  
$ compgen -b  
  
.  
:  
[  
alias  
bg  
bind  
break  
builtin  
caller  
cd  
command  
compgen  
complete  
compgen  
compgen  
continue  
declare  
dirs  
disown  
echo  
enable  
eval  
exec  
exit  
export  
false  
fc  
fg  
getopts  
hash  
help  
history  
jobs  
kill  
let  
local  
logout  
mapfile  
popd  
printf  
pushd  
pwd  
read  
readarray  
readonly  
return
```

```
set
shift
shopt
source
suspend
test
times
trap
true
type
typeset
ulimit
umask
unalias
unset
wait
```

Bakın burada listelenmiş olan tüm komutlar, bash kabuğu içerisinde dahili olarak bulunan araçları temsil eden komutlardır. Yani hangi sistemde çalışırsanız çalışın, bash kabuğunu kullanıyorsanız bu listedeki araçları da kullanabilirsiniz.

Harici Komutlar(External)

Harici komutlar ise, mevcut sistem üzerinde yüklü bulunan araçları çalıştırmamızı sağlayan komutlardır. Tabii ki bu tür komutlar harici olan araçları temsil eden komutlar olduğu için kullanmakta olduğunuz sisteme göre harici komutlar değişiklik gösterir. Örneğin siz komut satırı üzerinden metinleri düzenleyebilmenizi sağlayacak olan nano aracını çalıştırmak üzere kabuğa aracın ismini girdiğinizde eğer araç sistemde yüklü ise açılır. Eğer yüklü değilse komut yok hatası alırsınız. İşte burada girdiğiniz `nano` komutu harici bir komut olarak kabul ediliyor. Çünkü nano aracı bash kabuğunun içinde yüklü gelen bir araç değil, nano aracı harici olarak sisteme yüklenmiş olan bir metin editörü yazılımıdır.

Dahili ve harici komutlar arasındaki fark oldukça net gördüğünüz gibi. Dahili komutlar genelde temel sistem yönetimi için kullanılan en temel araçları temsil eden komutlardır. Bu sayede bash kabuğunun olduğu her yerden temel sistem yönetimi görevini yerine getirebiliyoruz. Örneğin sistemimiz çöktüğünde, sistemi kurtarma gibi işlemler için kabuğun dahili komutlarından faydalanabiliyoruz.

Bizler de eğitim içerisinde hem dahili komutlardan hem de sistem yönetimi için sıklıkla kullanılan harici araçlardan bahsediyor olacağız. Bu sayede temel sistem yönetimi için ihtiyaç duyacağınız araçlardan ve sistemin genel işleyişinden haberdar olabileceksiniz.

Dahili komutların aslında en temel işlevleri yerine getiren birtakım araçlar olduğundan bahsettiğimiz için, bu dahili komutların tüm kabuklarda ortak olarak bulunup bulunmadığı sorusu aklınıza gelmiş olabilir. Yani madem en temel araçlar kabuk içerisinde dahili komut olarak bulunuyor, bunlar tüm kabuklarda ortak olmalı değil mi ?

Tüm kabuklardaki tüm dahili komutlar birebir aynı olmasa da evet, alternatif kabuklarda da aşağı yukarı benzer dahili komutlar yer alıyor. Araçların isimleri veya kullanım biçimleri kabuktan kabuğa göre biraz farklılık gösterse de aslında tüm kabuklarda aşağı yukarı temel olarak benzer olan pek çok dahili komut bulunuyor. Yani aslında bash kabuğunu ve sistemin temel işleyişini öğrendiğinizde, gerektiğinde alternatif kabukların

dahili komutlarını da kısa sürede öğrenip kolayca kullanabilirsiniz. Zaten harici komutular yani sistemde yüklü bulunan araçları çalıştırmak için kullanılan komutlar tüm kabuklarda aynıdır. Örneğin sistemde nano aracı yüklü ise, **nano** komutunu girdiğimizde kabuk zsh da olsa bash de olsa nano aracı bulunup açılır çünkü bu araç sistemde yüklüdür.

Dolayısıyla bu eğitimin bash kabuğu üzerinden işleniyor olması sizleri kısıtlamıyor. Bash kabuğu dünya çapında en yaygın kullanıma sahip kabuktur. Zaten bu sebeple alternatif kabuklara oranla bash kabuğuyla çok daha sık karşılaşıyor olacaksınız. Ayrıca istisnalar hariç bir sistemde bash kabuğu varsayılan kabuk değilse bile, sistemde yüklü olacağı için daha önce ele aldığımız şekilde varsayılan kabuk olarak nasıl ayarlayabileceğinizi biliyorsunuz.

Anlatımların tam da bu noktasında, madem bash kabuğu en yaygın kullanıma sahip kabuk neden tüm dağıtımlar varsayılan olarak bash kabuğunu kullanmıyor diye düşünmüş olabilirsiniz.

Bash Neden Varsayılan Kabuk Değil ?

Çoğu zaman lisans veya etkileşimli kullanımda kolaylık gibi unsurlar, dağıtımların varsayılan olarak kullandığı kabukların farklı olmasına neden olabiliyor. Dağıtım özelinde neden bash değil de farklı bir kabuk tercih edildiğini, kullandığınız dağıtımın resmi dokümantasyonlarından kolayca öğrenebilirsiniz. Yani bu durum tek bir sebebi olmayabilir.

Örneğin MacOS lisans koşulları uymadığı için bash kabuğunun en son sürümünü kullanamıyordu. Mac, sisteminde yüklü gelen bash kabuk sürümü çok eski kalmaya başlayınca da lisans koşulları elverişli olan zsh kabuğuna geçiş yapmıştı. Bunun dışında Kali Linux dağıtımı da etkileşimli kullanımda kolay olduğu ve bash kabuğuna oranla daha fazla özelleştirme imkanı tanıdığı için zsh kabuğunu varsayılan kabuk olarak kullanmaya başladı. İşte tıpkı bu örnekler üzerinde olduğu gibi bash kabuğunun tercih edilmediği dağıtımlar ile karşılaşmanız mümkün. Yine de sizlerin de bildiği üzere aslında kullandığınız dağıtımın varsayılan kabuğunun hangisi olduğu çok da önemli değil. Zaten pek çok dağıtımda bash kabuğu ayrıca yüklü olacağı veya kendiniz kolayca yükleyebileceğiniz için tüm bu sistemlerde de bash kabuğuna rahatlıkla erişebilirsiniz.

Benim tüm eğitim boyunca bash kabuğu üzerinden ilerleyecek olma nedenim, daha yaygın kullanıma sahip olması. Yaygın kullanıma sahip bash kabuğunu öğrendiğinizde daha evrensel bir kabuğu öğrenmiş olacaksınız. Özellikle kabuk programlama konusunda geçmişten günümüze bash kabuğu en sık tercih edilen kabuk olduğu için pek çok sistemde geriye dönük uyumlulukları bozmamak için bash kabuğu kullanılmaya devam ediliyor. Dolayısıyla bash kabuğunu öğrenmek ve kullanmak hem kişisel hem de profesyonel anlamada çok mantıklı bir tercih.

Bu noktada metaforik olarak Bash kabuğunu kabuk dillerinin İngilizcesi olarak düşünebilirsiniz. Dünyada en yaygın konuşulan dilini öğrenmenin avantajları bash kabuğu için de kabuk dilleri bağlamında aynen geçerlidir. Özellikle kişiselden ziyade profesyonel alanda bash kabuğu alternatiflerine oranla yaygın kullanıma sahip. Zaten ileride bash kabuğunun neden tercih edildiğini bizzat kendiniz de fark edeceksiniz.

Lafı daha fazla uzatmak istemiyorum ancak lütfen bu eğitimin bash kabuğu üzerinden ele alındığının farkında olarak **eğitimi bash kabuğu üzerinden takip edin**. Eğer kullandığınız sistemde bash kabuğu varsayılan olarak kullanılan kabuk değilse lütfen varsayılan olarak ayarlayın. Aksi halde eğitimi takip etme noktasında sorun yaşamanız kaçınılmaz olur.

Bir sonraki bölümde de çok kısaca kabuğun bizim girdiğimiz komutları nasıl algılayıp değerlendirdiğinden bahsederek devam edebiliriz.

i Not: Eğer buradaki anlatımlar size sıkıcı veya gereksiz gibi geldiyse lütfen biraz daha sabırlı olun. Çünkü bu bölüm ve bir sonraki bölüm aslında tüm eğitim boyunca bahsedeceğimiz tüm konu ve kavramların temelini oluşturuyor. Temeli sağlam atamazsak üzerine inşa edeceğimiz her şey dayanaksız olur. O yüzden lütfen sabırlı olun ve yalnızca tamamlamış olmak için bu bölümleri geçmeyin. Bunun size hiç bir faydası olmaz. Gerekliyse bölümü baştan sonra tekrar edin veya ara verip başka zaman devam edin ancak lütfen buradaki temel bilgileri anlayarak eğitime devam edin.